

|                  |                                                                      |
|------------------|----------------------------------------------------------------------|
| Experiment No.   | 3                                                                    |
| Experiment Title | Regression Analysis using Linear and Regularized Models <sup>1</sup> |
| Student Name     | Nitin                                                                |
| Register Number  | 312224700142                                                         |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                         |
| Submission Date  | September 15, 2026                                                   |

## 1 Aim and Objective

To implement linear and regularized regression models (Linear, Ridge, Lasso, and Elastic Net) for predicting a continuous target variable – the sanctioned loan amount – evaluate their performance using multiple regression metrics, visualize model behavior, and analyze overfitting, underfitting, and bias–variance characteristics.

## 2 Dataset Description

|                          |                                                                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Dataset Name             | Loan Amount Prediction (Analytics Vidhya / Kaggle “Predict Loan Amount Data”)                                                         |
| Dataset Source           | Kaggle, originally hosted via the Analytics Vidhya Loan Prediction Practice Problem                                                   |
| Number of Samples (raw)  | 614 loan applications                                                                                                                 |
| Number of Samples (used) | 592 (after dropping 22 rows with missing target)                                                                                      |
| Number of Features       | 11 predictor features (4 numeric, 7 categorical) after dropping the identifier column                                                 |
| Target Variable          | LoanAmount (continuous, in thousands)                                                                                                 |
| Missing Values           | Present in Gender (13), Married (3), Dependents (15), Self_Employed (32), LoanAmount (22), Loan_Amount_Term (14), Credit_History (50) |
| Train–Test Split         | 80% / 20%, random_state=42                                                                                                            |

Feature list: Gender, Married, Dependents, Education, Self\_Employed, ApplicantIncome, CoapplicantIncome, Loan\_Amount\_Term, Credit\_History, Property\_Area, Loan\_Status. The Loan\_ID identifier column was dropped as it carries no predictive information.

## 3 Preprocessing Steps

- **Target rows with missing target:** The 22 rows with a missing LoanAmount were dropped, since the target cannot be imputed without introducing circular bias into a regression task.
- **Missing value handling (features):** Categorical features (Gender, Married, Dependents, Self\_Employed) were imputed with their mode; numeric features (Loan\_Amount\_Term, Credit\_History) were imputed with their median.

- **Encoding:** Dependents (ordinal, with “3+” mapped to 3) was kept numeric; the remaining categorical features (Gender, Married, Education, Self\_Employed, Property\_Area, Loan\_Status) were one-hot encoded with drop\_first=True to avoid the dummy-variable trap.
- **Feature scaling:** StandardScaler was fit on the training split only and applied to both train and test features before every model (Linear, Ridge, Lasso, Elastic Net), since regularization penalties are scale-dependent.
- **Train–test split:** train\_test\_split with test\_size=0.2, random\_state=42.

## 4 Exploratory Data Analysis

The reusable function `generate_eda_summary()` from `ml_lab_utils.py` (developed in Experiment 1, unmodified) was used to produce the consolidated 12-panel EDA figure below.

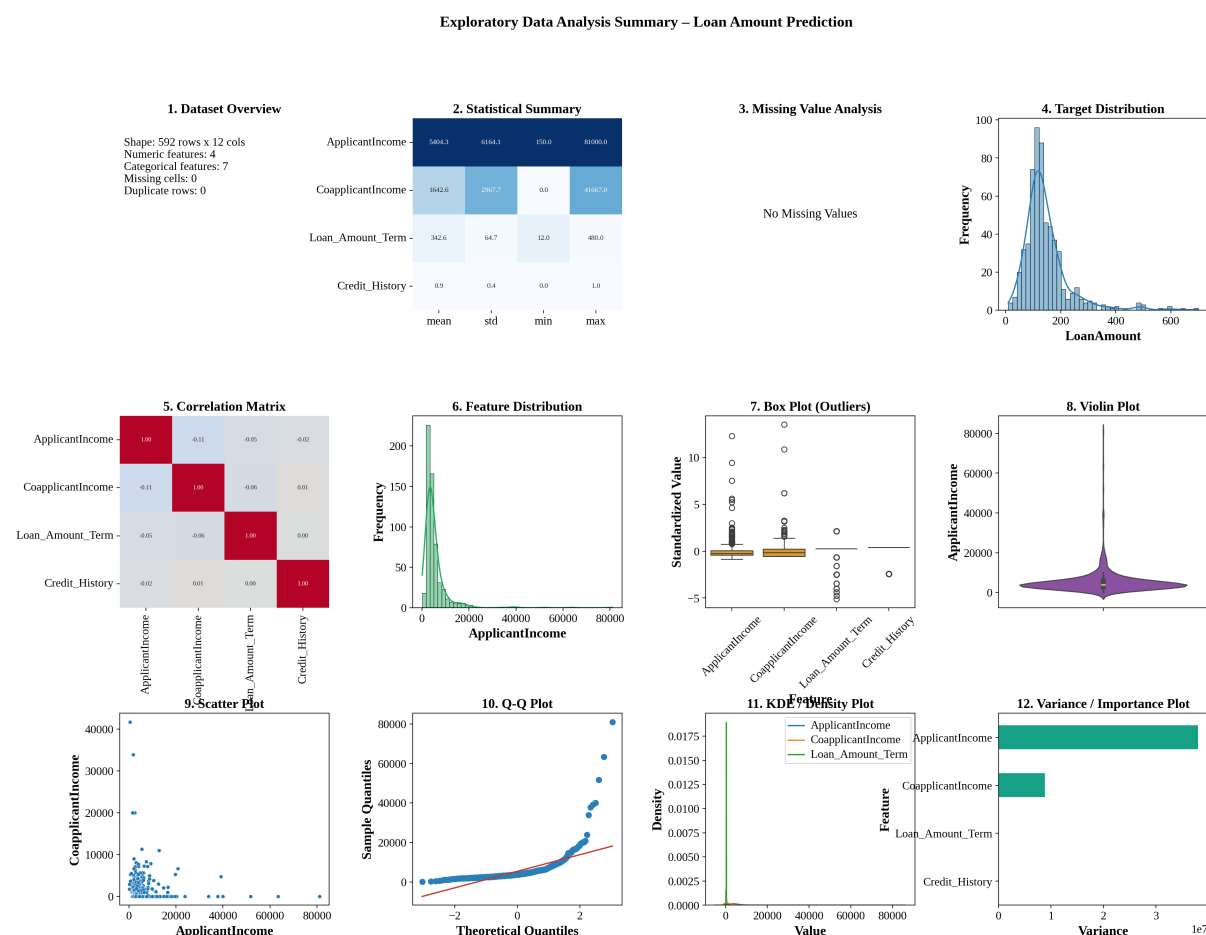


Figure 1: Consolidated 12-panel EDA summary for the Loan Amount Prediction dataset.

**Inference:** After imputation there are no remaining missing values. **Applicant Income** is the highest-variance numeric feature by a wide margin and is strongly right-skewed (Panels 6, 9, 10, 12), with a small number of very high-income applicants visible as outliers in the box plot (Panel 7) and as the far-right point in the Q–Q plot’s upper tail

(Panel 10). `LoanAmount` itself (Panel 4) is also right-skewed, which is consistent with the residual patterns observed later in Section 6. `ApplicantIncome` and `CoapplicantIncome` show a weak negative correlation (Panel 5,  $-0.11$ ), suggesting the two income sources are largely independent (i.e., loan applications are not being systematically structured to trade off one income source against the other).

## Additional Required Visualizations

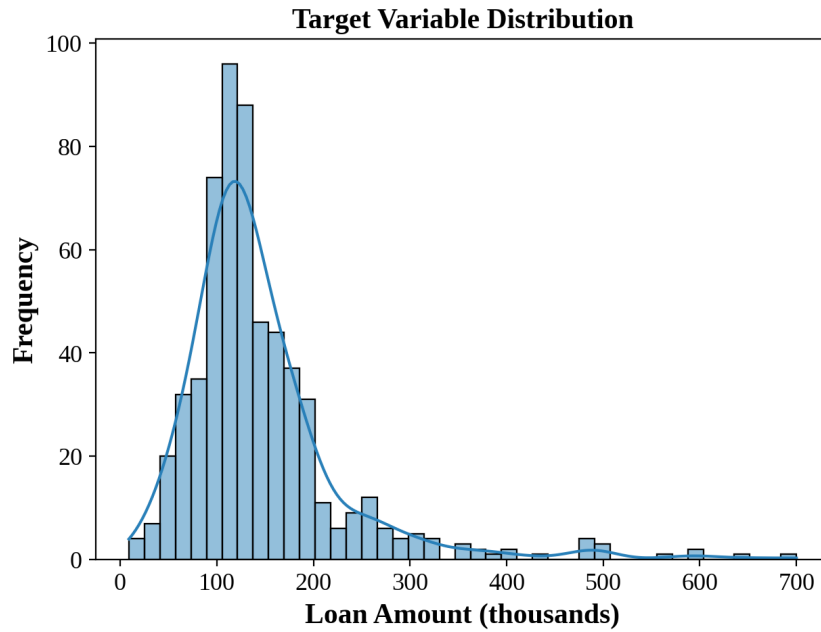


Figure 2: Target variable (`LoanAmount`) distribution.

**Inference:** The target is unimodal and right-skewed, with most sanctioned loan amounts falling between 100 and 170 (thousand), and a long tail of high-value loans extending past 400 – these tail cases are the hardest for a linear model to predict accurately, as confirmed in the predicted-vs-actual plots (Figure 4).

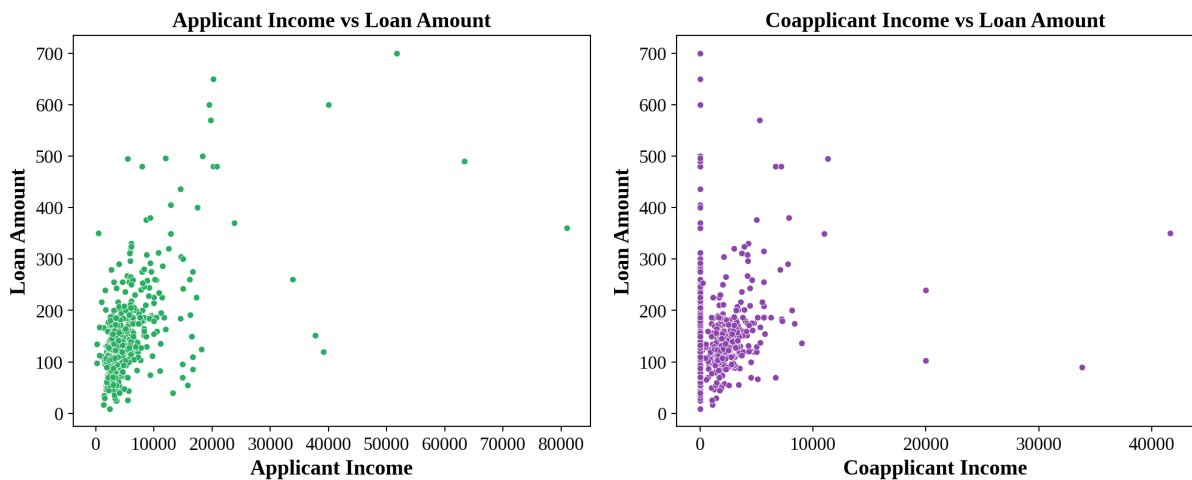


Figure 3: Applicant income and coapplicant income vs. loan amount.

**Inference:** Both income features show a weak positive, high-variance relationship with loan amount – higher income allows for (but does not guarantee) a larger loan, consistent with the low  $R^2$  values obtained later in Section 6. Applicant income shows a slightly stronger and more visible upward trend than coapplicant income.

## 5 Implementation Details

**Linear Regression** models the relationship between input features and the continuous target as  $\hat{y} = \beta^\top \mathbf{x} + \beta_0$ , fit by minimizing squared error. It is simple, interpretable, and serves as this experiment’s baseline.

**Ridge Regression** adds an  $L_2$  penalty  $\alpha \sum_j \beta_j^2$  to the ordinary least-squares objective, shrinking coefficient magnitudes toward zero (but not exactly to zero) and reducing variance at the cost of a small increase in bias.

**Lasso Regression** adds an  $L_1$  penalty  $\alpha \sum_j |\beta_j|$ , which can shrink coefficients exactly to zero, performing automatic feature selection.

**Elastic Net** combines both penalties,  $\alpha \left( \text{ll\_ratio} \sum_j |\beta_j| + \frac{1-\text{ll\_ratio}}{2} \sum_j \beta_j^2 \right)$ , balancing Ridge’s stability with Lasso’s sparsity.

Listing 1: Reusable regression training call, from `ml_lab_utils.py` (Experiment 1) applied to the baseline model.

```

1 baseline_models = {"Linear Regression": LinearRegression()}
2 baseline_results_df, baseline_fitted = train_evaluate_regression(
3     baseline_models, X_train_scaled, X_test_scaled, y_train,
4     y_test,
5     scale=False)

```

Listing 2: Hyperparameter search space and GridSearchCV/RandomizedSearchCV tuning, from `experiment3_loan_regression.py`.

```

1 search_spaces = {
2     "Ridge Regression": (Ridge(random_state=42),
3                           {"alpha": [0.01, 0.1, 1, 10, 100]}),
4     "Lasso Regression": (Lasso(random_state=42, max_iter=10000),
5                           {"alpha": [0.001, 0.01, 0.1, 1, 10]}),
6     "Elastic Net Regression": (ElasticNet(random_state=42,
7                                             max_iter=10000),
8                                {"alpha": [0.01, 0.1, 1, 10],
9                                "l1_ratio": [0.2, 0.5, 0.8]}),
9 }
10 gs = GridSearchCV(estimator, grid, cv=KFold(5, shuffle=True,
11                                             random_state=42),
12                   scoring="r2", n_jobs=-1)
13 gs.fit(X_train_scaled, y_train)

```

## 6 Experimental Setup

|                      |                                     |
|----------------------|-------------------------------------|
| Python Version       | 3.12                                |
| Scikit-Learn Version | 1.8.0                               |
| IDE                  | Jupyter Notebook / VS Code          |
| Operating System     | Ubuntu 24.04 LTS                    |
| Hardware             | Standard lab workstation (CPU only) |

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

## 7 Hyperparameter Tuning Results

Table 1: Hyperparameter Tuning Summary

| Model                  | Search Method | Best Parameters              | Best CV $R^2$ |
|------------------------|---------------|------------------------------|---------------|
| Ridge Regression       | Grid / Random | $\alpha = 100$               | 0.3714        |
| Lasso Regression       | Grid / Random | $\alpha = 10$                | 0.3311        |
| Elastic Net Regression | Grid / Random | $\alpha = 1$ , l1_ratio= 0.8 | 0.3707        |

GridSearchCV and RandomizedSearchCV selected *identical* best hyperparameters and CV  $R^2$  for all three regularized models on this small search space, with RandomizedSearchCV requiring modestly fewer fit calls (Ridge: 5 vs. 5, Lasso: 5 vs. 5, Elastic Net: 10 vs. 12), since the grids here are small enough that random sampling covers most or all of the space.

## 8 Performance Tables

Table 2: Cross-Validation Performance ( $K = 5$ , on training data)

| Model                  | MAE   | MSE     | RMSE  | $R^2$         |
|------------------------|-------|---------|-------|---------------|
| Linear Regression      | 43.11 | 5508.55 | 74.22 | 0.3109        |
| Ridge Regression       | 44.08 | 4935.28 | 70.25 | <b>0.3714</b> |
| Lasso Regression       | 46.23 | 5281.51 | 72.67 | 0.3311        |
| Elastic Net Regression | 43.88 | 4944.29 | 70.32 | 0.3707        |

Table 3: Test Set Performance

| Model                  | MAE   | MSE     | RMSE  | $R^2$         |
|------------------------|-------|---------|-------|---------------|
| Linear Regression      | 36.88 | 3335.43 | 57.75 | 0.0952        |
| Ridge Regression       | 37.55 | 3159.55 | 56.21 | 0.1429        |
| Lasso Regression       | 37.80 | 3176.80 | 56.36 | 0.1382        |
| Elastic Net Regression | 37.43 | 3151.21 | 56.14 | <b>0.1452</b> |

All four models train in well under a millisecond on this dataset size (592 samples, 12 features); training time is therefore not a practically distinguishing factor here, and all further discussion focuses on predictive performance.

## Test Set Performance Comparison

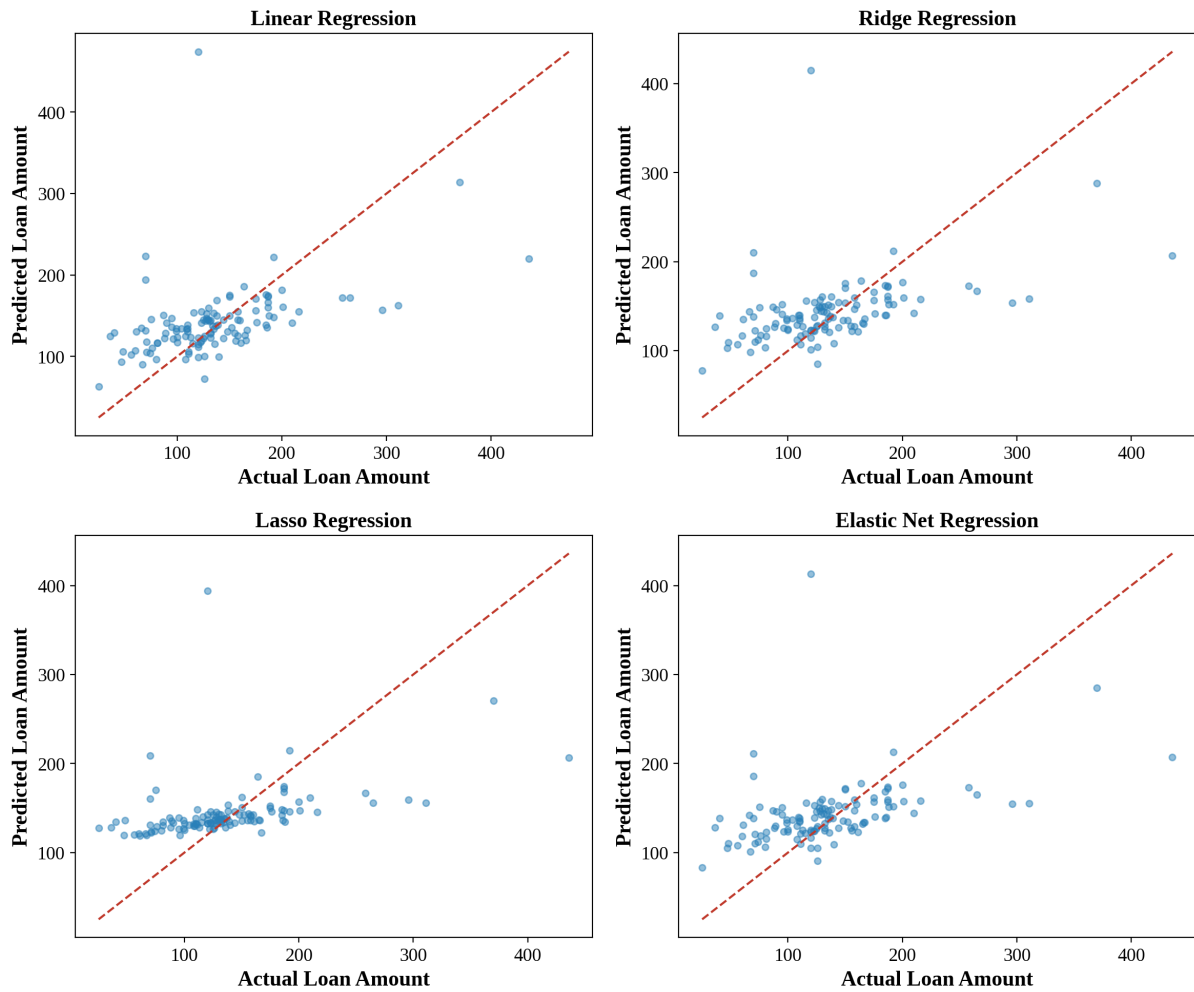


Figure 4: Predicted vs. actual loan amount for all four models on the held-out test set.

**Inference:** All four models show the same qualitative failure mode: they compress predictions toward the mean and systematically under-predict the small number of very large loan amounts (the points far to the right, well below the dashed  $y = x$  line). Ridge and Elastic Net show slightly tighter clustering around the diagonal for mid-range loan amounts than plain Linear Regression, consistent with their higher test  $R^2$ .

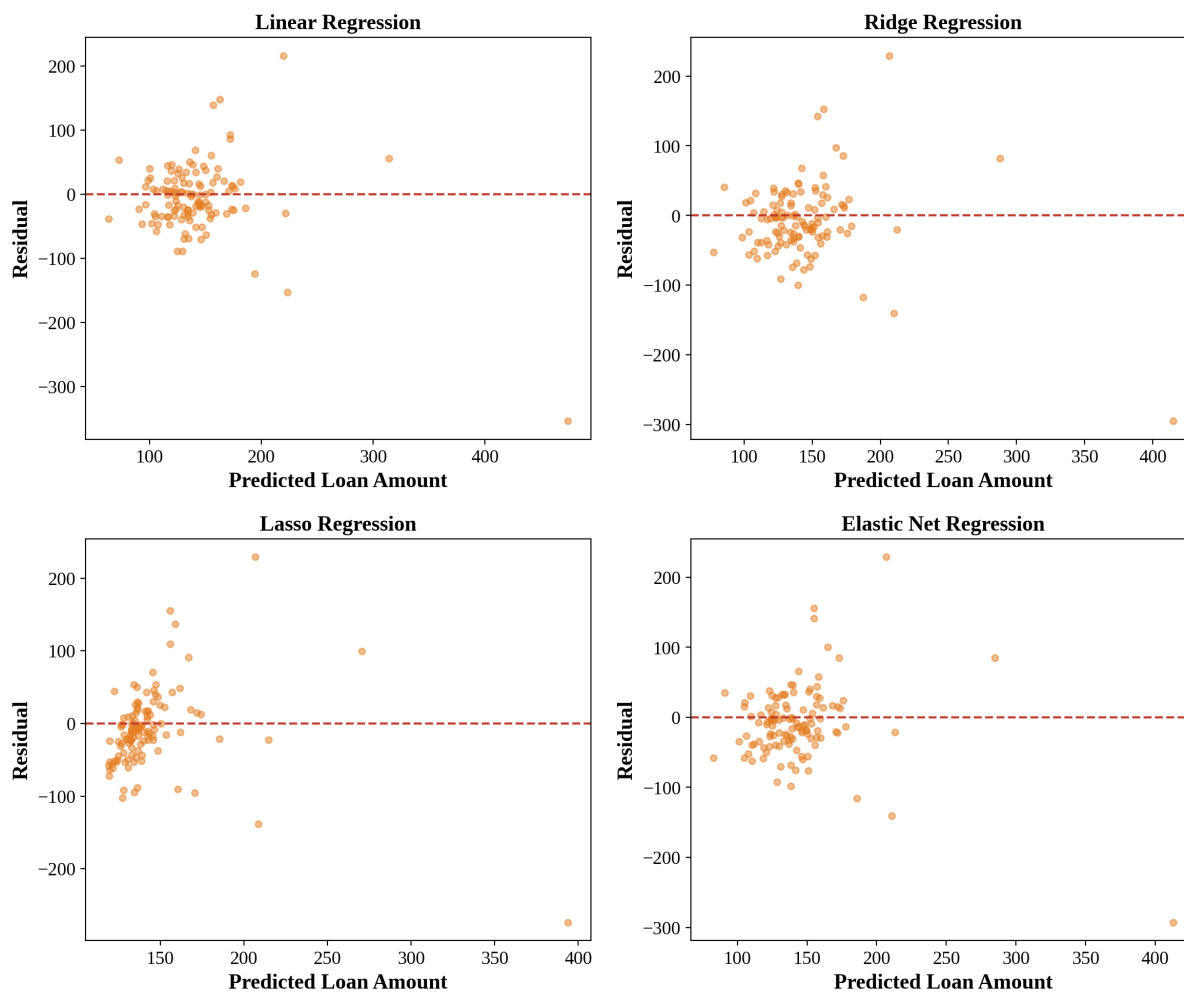


Figure 5: Residual plots (residual = actual – predicted) for all four models.

**Inference:** Residuals are not randomly scattered around zero – there is a clear funnel/fan pattern (heteroscedasticity), with residual variance increasing for larger predicted loan amounts. This, combined with the positive residuals concentrated at higher predicted values, confirms that a plain linear model under-fits the (right-skewed, likely non-linear) relationship between income and loan amount at the high end of the distribution – a log-transform of the target would be a natural next step to address this in future work.

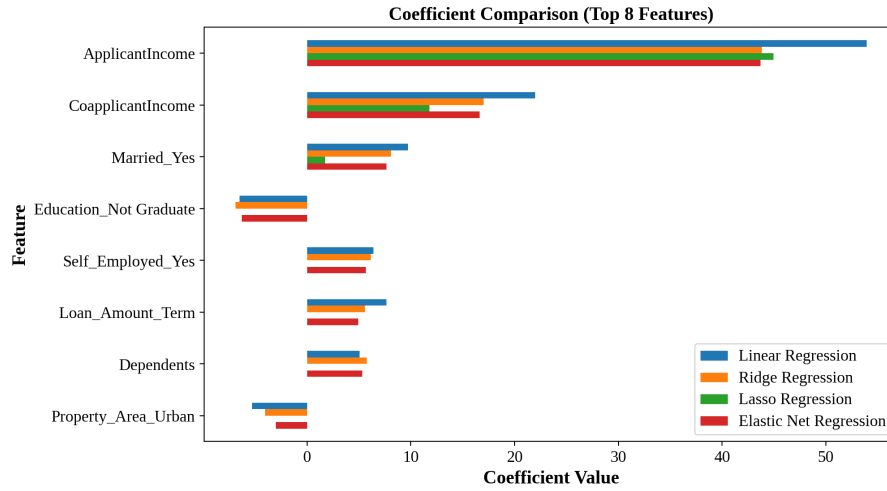


Figure 6: Coefficient comparison across Linear, Ridge, Lasso, and Elastic Net (top 8 features by summed magnitude).

**Table 4: Coefficient Comparison (top 3 features)**

| Feature           | Linear | Ridge | Lasso | Elastic Net |
|-------------------|--------|-------|-------|-------------|
| ApplicantIncome   | 53.95  | 43.83 | 44.98 | 43.72       |
| CoapplicantIncome | 21.97  | 17.00 | 11.79 | 16.63       |
| Married_Yes       | 9.72   | 8.08  | 1.74  | 7.64        |

**Inference:** `ApplicantIncome` is the dominant predictor across every model. Ridge shrinks all coefficients smoothly toward zero without eliminating any of them. Lasso is far more aggressive: it drives `Dependents`, `Loan_Amount_Term`, `Credit_History`, `Gender_Male`, `Self_Employed_Yes`, and three `Property_Area/Loan_Status` dummies to *exactly* zero, keeping only `ApplicantIncome`, `CoapplicantIncome`, and `Married_Yes` with non-trivial weight. Elastic Net sits between the two, shrinking coefficients but retaining more non-zero features than Lasso.



## 9 Overfitting and Underfitting Analysis

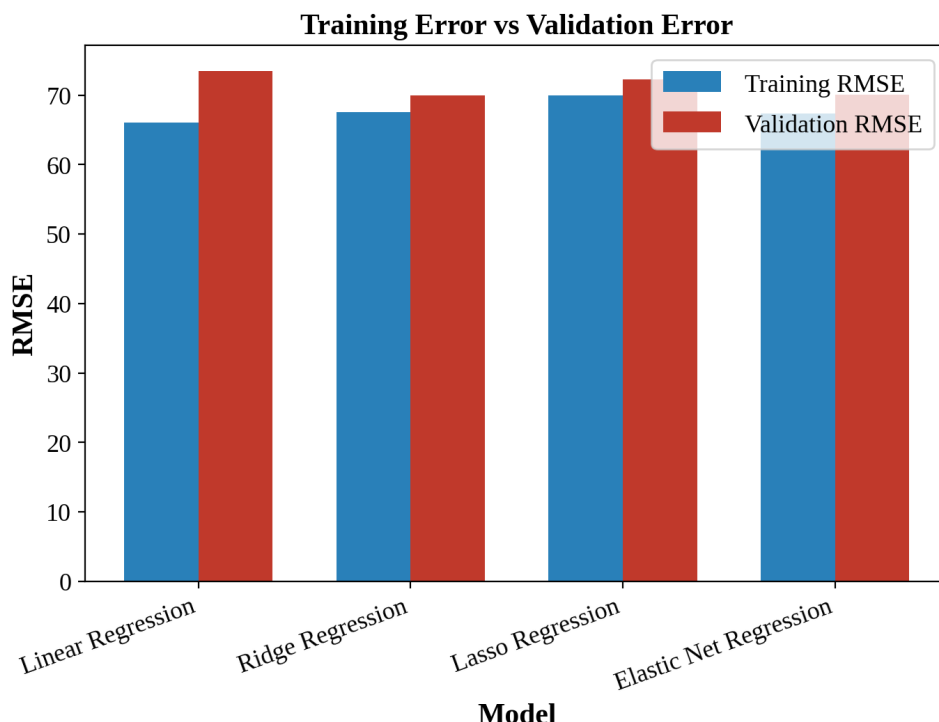


Figure 7: Training RMSE vs. 5-fold validation RMSE for all four models.

**Training vs. validation error:** Plain Linear Regression has the lowest training RMSE (66.09) but the *largest* training–validation gap (66.09  $\rightarrow$  73.52, a gap of 7.43), the classic signature of a model fitting noise in the training data (mild overfitting relative to the regularized alternatives). Ridge (67.60  $\rightarrow$  69.94, gap 2.35) and Elastic Net (67.34  $\rightarrow$  70.02, gap 2.69) both substantially narrow this generalization gap at the cost of a small increase in training error – exactly the bias-variance trade-off regularization is designed to make. Lasso narrows the gap less well here (69.99  $\rightarrow$  72.30, gap 2.31 in RMSE terms but from a higher starting point), consistent with it having discarded several features (`Credit_History` in particular) that do carry real, if modest, predictive signal.

**Effect of regularization strength:** Figure 8 shows Ridge’s regularization path directly: as  $\alpha$  increases from 0.01 to 100, training RMSE rises only slightly (66.09  $\rightarrow$  67.60) while validation RMSE falls substantially (73.52  $\rightarrow$  69.94). This is the textbook regularization effect – controlled bias increase buys a larger variance reduction, improving generalization, up to the point where too much regularization would eventually push training error up faster than validation error falls (not yet reached within the tested  $\alpha$  range here, since  $\alpha = 100$  was selected as the CV-optimal value by GridSearchCV).

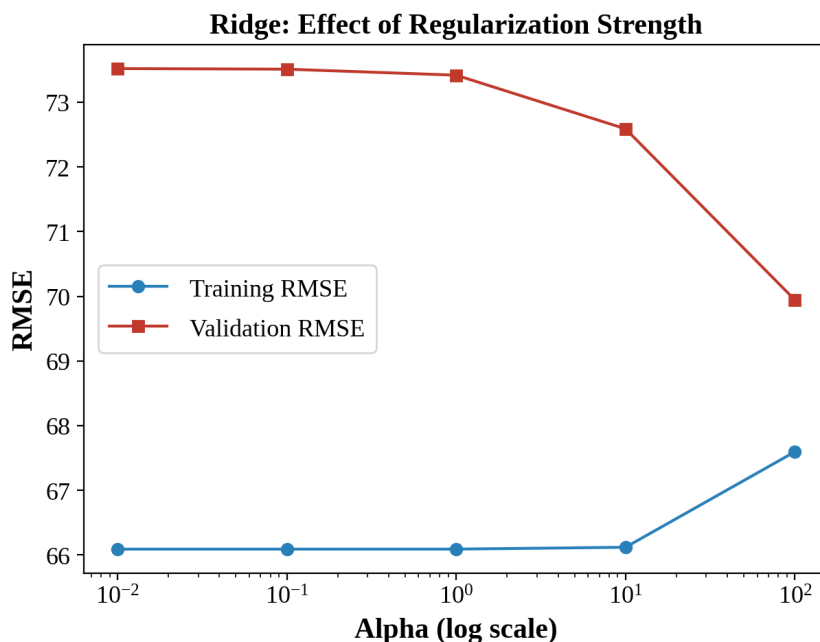


Figure 8: Ridge: training vs. validation RMSE as a function of  $\alpha$ .

**Improvement in generalization after tuning:** Cross-validated  $R^2$  improved from 0.3109 (untuned Linear Regression) to 0.3714 (tuned Ridge) – a meaningful relative improvement of roughly 19%, achieved purely by adding an appropriately-tuned  $L_2$  penalty with no change to the feature set.

## 10 Bias–Variance Analysis

**Bias behavior of Linear Regression:** Ordinary Linear Regression has the lowest bias among the four models (it best fits the training data, as shown by its lowest training RMSE), but this low bias comes at the cost of higher variance – it is the most sensitive to the particular training fold, producing the largest train–validation gap observed in this experiment.

**Variance reduction using Ridge and Elastic Net:** Both Ridge and Elastic Net shrink coefficient magnitudes (Figure 6), which directly reduces the model’s sensitivity to small changes in the training data (variance). This is empirically confirmed by their smaller train–validation RMSE gaps (2.35 and 2.69 respectively) compared to plain Linear Regression’s gap of 7.43, and by their higher cross-validated  $R^2$  (0.3714 and 0.3707 vs. 0.3109).

**Feature sparsity effect in Lasso:** Figure 9 shows the number of non-zero coefficients falling from 12 (at  $\alpha = 0.001$ ) to just 3 (at  $\alpha = 10$ , the CV-selected optimum) as regularization strength increases. This sparsity is a form of automatic feature selection: Lasso concludes that only `ApplicantIncome`, `CoapplicantIncome`, and `Married_Yes` carry enough signal to justify a non-zero coefficient at its chosen  $\alpha$ , trading a small amount of additional bias (its test  $R^2$  of 0.1382 is the lowest of the three regularized models) for a much simpler, more interpretable model.

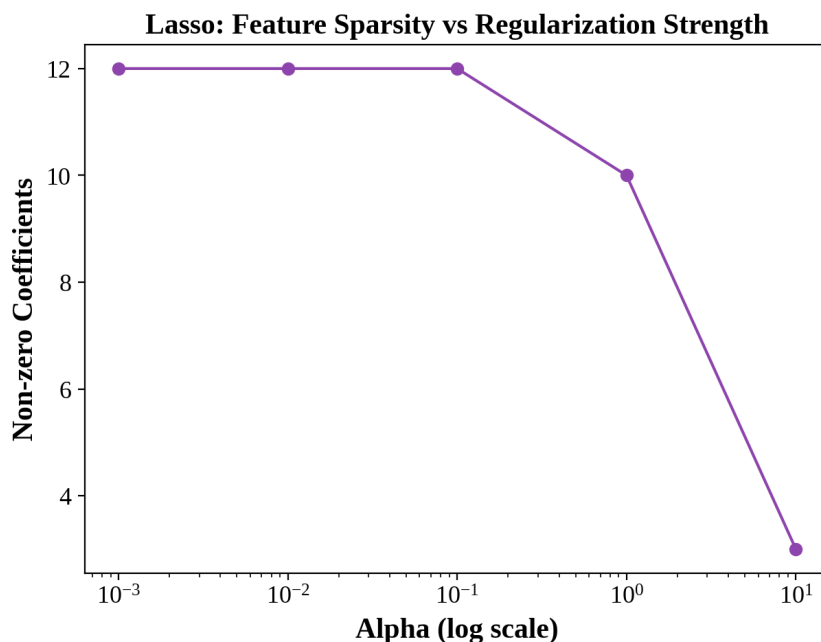


Figure 9: Lasso: number of non-zero coefficients vs. regularization strength  $\alpha$ .

## 11 Observations and Conclusion

All four models achieve broadly similar test-set performance (test  $R^2$  between 0.095 and 0.145), reflecting a genuinely difficult regression target: loan amount sanctioned depends on underwriting factors (e.g., detailed credit assessment, property valuation) not captured in this feature set, and the target itself is right-skewed with influential high-value outliers. Among the four models, **Elastic Net Regression** achieved the best test-set  $R^2$  (0.1452) and joint-best RMSE (56.14), narrowly ahead of Ridge (0.1429, 56.21); both clearly outperform plain Linear Regression (0.0952, 57.75) and Lasso (0.1382, 56.36). The optimal hyperparameters – Ridge  $\alpha = 100$ , Lasso  $\alpha = 10$ , Elastic Net  $\alpha = 1$  with `l1_ratio=0.8` – were each confirmed identically by both `GridSearchCV` and `RandomizedSearchCV`, giving confidence that these are true optima within the specified search space rather than artifacts of a particular search strategy. In terms of the accuracy-versus-complexity trade-off: Elastic Net offers the best accuracy but retains most features with shrunk (not zeroed) weights; Lasso sacrifices a small amount of accuracy for a much simpler 3-feature model, which may be preferable in a setting where model interpretability or deployment simplicity outweighs the last few percentage points of  $R^2$ . For this dataset, given the modest improvement in test  $R^2$  (0.143 vs 0.145) between Ridge and Elastic Net, either would be a reasonable choice; the sparser Lasso model is the more attractive option if explainability to loan officers is a priority, despite its slightly lower accuracy.

## 12 References

1. Scikit-learn Developers, “Linear Models,” scikit-learn documentation. [Online]. Available: [https://scikit-learn.org/stable/modules/linear\\_model.html](https://scikit-learn.org/stable/modules/linear_model.html)
2. Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” scikit-

learn documentation. [Online]. Available: [https://scikit-learn.org/stable/modules/grid\\_search.html](https://scikit-learn.org/stable/modules/grid_search.html)

3. Analytics Vidhya / Kaggle, “Loan Prediction Problem Dataset.” [Online]. Available: <https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset>
4. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.